Python Library Documentation: module heapq

NAME

heapq - Heap queue algorithm (a.k.a. priority queue).

MODULE REFERENCE

https://docs.python.org/3.14/library/heapq.html

The following documentation is automatically generated from the Python source files. It may be incomplete, incorrect or include features that are considered implementation detail and may vary between Python implementations. When in doubt, consult the module reference at the location listed above.

DESCRIPTION

Heaps are arrays for which a[k] <= a[2*k+1] and a[k] <= a[2*k+2] for all k, counting elements from 0. For the sake of comparison, non-existing elements are considered to be infinite. The interesting property of a heap is that a[0] is always its smallest element.

Usage:

```
heap = []            # creates an empty heap
heappush(heap, item) # pushes a new item on the heap
item = heappop(heap) # pops the smallest item from the heap
item = heap[0]       # smallest item on the heap without popping it
heapify(x)           # transforms list into a heap, in-place, in linear time
item = heappushpop(heap, item) # pushes a new item and then returns
# the smallest item; the heap size is unchanged
item = heapreplace(heap, item) # pops and returns smallest item, and adds
# new item; the heap size is unchanged
```

Our API differs from textbook heap algorithms as follows:

- We use 0-based indexing. This makes the relationship between the index for a node and the indexes for its children slightly less obvious, but is more suitable since Python uses 0-based indexing.
- Our heappop() method returns the smallest item, not the largest.

These two make it possible to view the heap as a regular Python list without surprises: heap[0] is the smallest item, and heap.sort() maintains the heap invariant!

FUNCTIONS

heapify(heap, /)

Transform list into a heap, in-place, in O(len(heap)) time.
heappop(heap, /)
Pop the smallest item off the heap, maintaining the heap invariant.
heappush(heap, item, /)
Push item onto heap, maintaining the heap invariant.
heappushpop(heap, item, /)
Push item on the heap, then pop and return the smallest item from the heap.
The combined action runs more efficiently than heappush() followed by
a separate call to heappop().
heapreplace(heap, item, /)
Pop and return the current smallest value, and add the new item.
This is more efficient than heappop() followed by heappush(), and can be
more appropriate when using a fixed-size heap. Note that the value
returned may be larger than item! That constrains reasonable uses of
this routine unless written as part of a conditional replacement:
if item > heap[0]:
item = heapreplace(heap, item)
merge(*iterables, key=None, reverse=False)
Merge multiple sorted inputs into a single sorted output.
Similar to sorted(itertools.chain(*iterables)) but returns a generator,
does not pull the data into memory all at once, and assumes that each of
the input streams is already sorted (smallest to largest).
>>> list(merge([1,3,5,7], [0,2,4,8], [5,10,15,20], [], [25]))
[0, 1, 2, 3, 4, 5, 5, 7, 8, 10, 15, 20, 25]
If *key* is not None, applies a key function to each element to determine
its sort order.
>>> list(merge(['dog', 'horse'], ['cat', 'fish', 'kangaroo'], key=len))
['dog', 'cat', 'fish', 'horse', 'kangaroo']
nlargest(n, iterable, key=None)
Find the n largest elements in a dataset.
Equivalent to: sorted(iterable, key=key, reverse=True)[:n]
nsmallest(n, iterable, key=None)
Find the n smallest elements in a dataset.
Equivalent to: sorted(iterable, key=key)[:n]
DATA
__about

__ = 'Heap queues\n\n[explanation by Fran ois Pinard]\n\nH... t...
__all__ = ['heappush', 'heappop', 'heapify', 'heapreplace', 'merge', '...
FILE
c:\python314\lib\heapq.py